

```

/*
Logique
Récursivité : Utiliser une méthode récursive pour explorer toutes les prises possibles à partir d'une position
donnée.
Liste des rafles : Maintenir une liste temporaire pour chaque chemin de rafle, qui contient les cases sautées
(ou les paires (Prise, Arrivée)).
Validation des conditions : À chaque saut, vérifier si des prises supplémentaires sont possibles depuis la
nouvelle position.
Retour des résultats : La méthode doit retourner toutes les séquences de rafles valides.
*/

```

```

public static (List<int> DeplacementsSimples, List<List<(int Prise, int Arrivee)>> Rafles)
ObtenirMouvementsPossibles(int numeroCaseManoury)
{
    List<int> deplacementsSimples = new List<int>();
    List<List<(int Prise, int Arrivee)>> rafles = new List<List<(int, int)>>();

    // Directions : Nord-Ouest, Nord-Est, Sud-Ouest, Sud-Est
    int[] directions = { -11, -9, 9, 11 };

    int numeroCaseBox = GestionDamier.ObtenirIndicePictureBox(numeroCaseManoury);
    (int ligne, int colonne) = GestionDamier.IndicesVersCoordonnees(numeroCaseBox);

    if (!GestionDamier.DamierContenu[ligne, colonne].EstVide())
    {
        // Déplacements simples
        foreach (var direction in directions)
        {
            int nouveaunumeroCaseBox = numeroCaseBox + direction;
            (int nouvelleLigne, int nouvelleColonne) =
GestionDamier.IndicesVersCoordonnees(nouveaunumeroCaseBox);

            if (GestionDamier.EstNumeroPictureBoxValide(nouveaunumeroCaseBox))
            {
                var contenuCase = GestionDamier.DamierContenu[nouvelleLigne, nouvelleColonne];
                if (contenuCase.TypePiece == LogiqueMouvementsDames.TypePiece.Vide &&
                    ((BrunoInterfaceGraphiqueDames.CouleurPieceCliquee == "Blanc" && direction < 0) ||

```

```

        (BrunoInterfaceGraphiqueDames.CouleurPieceCliquee == "Noir" && direction > 0)))
    {
        deplacementsSimples.Add(GestionDamier.PictureBoxVersManoury[nouveaunumeroCaseBox]);
    }
}

// Détection des rafles (prises multiples)
void RechercherRafles(int caseActuelle, List<(int Prise, int Arrivee)> cheminActuel)
{
    bool prisePossible = false;

    foreach (var direction in directions)
    {
        int caseCible = caseActuelle + direction;
        (int ligneCible, int colonneCible) = GestionDamier.IndiceVersCoordonnees(caseCible);

        if (GestionDamier.EstNumeroPictureBoxValide(caseCible))
        {
            var contenuCible = GestionDamier.DamierContenu[ligneCible, colonneCible];

            // Vérifier si la case contient une pièce adverse
            if (contenuCible.TypePiece != LogiqueMouvementsDames.TypePiece.Vide &&
                contenuCible.CouleurPiece !=
ConvertirCouleurTrait(BrunoInterfaceGraphiqueDames.CouleurPieceCliquee))
            {
                int caseSaut = caseCible + direction;
                (int ligneSaut, int colonneSaut) = GestionDamier.IndiceVersCoordonnees(caseSaut);

                if (GestionDamier.EstNumeroPictureBoxValide(caseSaut) &&
                    GestionDamier.DamierContenu[ligneSaut, colonneSaut].TypePiece ==
LogiqueMouvementsDames.TypePiece.Vide)
                {
                    prisePossible = true;
                    List<(int Prise, int Arrivee)> nouveauChemin = new List<(int Prise, int
Arrivee)>(cheminActuel)
                    {

```

```

        (GestionDamier.PictureBoxVersManoury[caseCible],
GestionDamier.PictureBoxVersManoury[caseSaut])
        };

        // Marquer la case cible comme temporairement vide pour éviter des boucles infinies
        var pieceTemporaire = GestionDamier.DamierContenu[ligneCible, colonneCible];
        GestionDamier.DamierContenu[ligneCible, colonneCible] = new CaseVide();

        // Explorer les prises suivantes
        RechercherRafles(caseSaut, nouveauChemin);

        // Restaurer la pièce
        GestionDamier.DamierContenu[ligneCible, colonneCible] = pieceTemporaire;
    }
}

}

if (!prisePossible && cheminActuel.Count > 0)
{
    // Ajouter le chemin complet si aucune autre prise n'est possible
    rafles.Add(cheminActuel);
}

}

// Lancer la recherche des rafles à partir de la position initiale
RechercherRafles(numeroCaseBox, new List<(int Prise, int Arrivee)>());
}

return (deplacementsSimples, rafles);
}

/*
Rafles :

```

Une fonction récursive RechercherRafles est utilisée pour explorer toutes les prises possibles.
À chaque saut, une nouvelle liste est créée pour suivre le chemin des cases prises.
Simulation temporaire :

La case de la pièce adverse est temporairement marquée comme vide pour éviter des boucles infinies.
Arrêt de la récursion :

Si aucune autre prise n'est possible, le chemin actuel est ajouté à la liste des rafles.

Retour de la méthode

Déplacements simples : Une liste des cases accessibles directement.

Rafles : Une liste de listes, chaque sous-liste contenant les paires (Prise, Arrivée) pour un chemin de rafle.

Ce code gère à la fois les déplacements simples et les prises multiples.

*/